

Impact, Technical & Service Design

Three assessments of Equina as it actually stands on 25 August 2026 — what it would do to users, how it is built, and the service that has to exist behind it.

Read section 3 of Part I first. Two critical findings appear in no earlier document: the backend exists on one disk with no version-control baseline, and an exposed Supabase secret key has not been rotated.

BACKEND FOUNDATION

7.3/10

was 5.5 pre-Sprint 6

HOSTED READINESS

5.9/10

was 3.8 pre-Sprint 6

MIGRATIONS SYNCED

34/34

lint clean, all additive

EDGE FUNCTIONS LIVE

6/29

23 implemented, undeployed

WORKERS SCHEDULED

1/5

storage cleanup only

CAPABILITIES ON

0/10

auth only · fail-closed

Assessed 2026-08-25
Source Equina 2.0 – copie
Basis repository inspection + 2026-07-29 backend audit

Project vdcrllyzxbdjonotrujb · Frankfurt
Client equina-five.vercel.app
Verified npm test · npm run typecheck pass

Contents

PART I

Impact Assessment

What changes, what can go wrong, and what must be true before anything is switched on.

1	Purpose And Scope	7	Operational Impact
2	Current State Summary	8	Business And Commercial Impact
3	Critical Finding: The Work Is Not Under Version Control	9	Consolidated Risk Register
4	User Impact	10	Impact Of Doing Nothing
5	Data Protection Impact	11	Recommendation
6	Security Impact	12	Sign-Off

PART II

Technical Design

The system as it is actually built, the two structural problems in it, and the target architecture.

1	Goals And Constraints	8	Environments And Deployment
2	System Overview	9	Test And Verification Strategy
3	The Three-Layer Activation Gate	10	Observability Design
4	Client Architecture	11	Known Technical Debt
5	Backend Architecture	12	Decision Log
6	Structural Problem 1: The Dual Data Stack	13	Immediate Technical Priorities
7	Structural Problem 2: The App.tsx Monolith		

PART III

Service Design

The whole service — the app, the humans behind it, and the moments where it can fail a rider.

1	Service Proposition	6	Moments Of Truth
2	Service Principles	7	Channel Design
3	Actors	8	The Missing Operating Model
4	Service Ecosystem Map	9	Service Readiness Matrix
5	Service Blueprints	10	Service Recommendations

Impact Assessment

What changes, what can go wrong, and what must be true
before anything is switched on.

ASSESSMENT DATE	2026-08-25
ASSESSED ARTIFACT	Equina 2.0 - copie working tree, Supabase project vdcrllyzxbdjonotrujb (Frankfurt), web client https://equina-five.vercel.app
ASSESSMENT BASIS	repository inspection, docs/EQUINA_BACKEND_AUDIT_2026-07-29.md , docs/EQUINA_BACKEND_DEPLOYMENT_INVENTORY.md , docs/EQUINA_ACTION_REGISTRY.md , and a re-run of the automated suite on 2026-08-25.

1 Purpose And Scope

This assessment answers three questions before any further rollout decision:

1. What would actually change for users, the business, and operations if Equina moved from its current internal state toward staged production?
2. What harm is possible today, in the state the system is in right now?
3. What must be true before each capability may be turned on?

Scope covers the mobile/web client, the Supabase backend (database, RLS, Storage, Edge Functions, schedulers), the hosted web deployment, and the operating processes around them. It does not cover marketing, pricing execution, or App Store commercial terms.

Assessment Vocabulary

The audit vocabulary is retained because impact depends on it:

- **Implemented** — reviewed code and migrations exist in the repository.
- **Deployed** — the exact artifact runs in the linked Supabase project.
- **Configured** — every required secret, redirect, scheduler, and policy is present.
- **Proven** — the journey passed remote integration and physical-device tests.

Impact is only realized when all four are true. Most of Equina is *implemented*; very little is *proven*.

2 Current State Summary

Verified on 2026-08-25 against the working tree:

DIMENSION	STATE
Automated suite	npm test passes (core flows, backend security contracts, isolated Postgres migrations)
Type safety	npm run typecheck passes with no errors
Migrations	34 additive migrations, all synchronized to the linked project
Edge Functions	29 implemented; 6 deployed (capabilities, 4 Horse/Records upload/delete boundaries, Storage cleanup)
Schedulers	1 of 5 active (equina-storage-cleanup, every 5 minutes)
Capability response	auth: true; all 10 privileged product capabilities false
Live users	1 named internal user, Horse + Records only, 30-day override

DIMENSION	STATE
Backend foundation score	~7.3/10 (from ~5.5/10 pre-Sprint 6)
Hosted production readiness	~5.9/10 (from ~3.8/10 pre-Sprint 6)
Environments	1 (production-linked). Staging not provisioned
Version control baseline	1 commit, 28 files, dated 2026-06-01

The fail-closed capability posture is correct and is the main reason user-facing harm is currently near zero. The product cannot claim account operations, Ralf, Club publishing, listing creation, messaging, push, or checkout are live, because they are not.

3 Critical Finding: The Work Is Not Under Version Control

This is the highest-severity finding in this assessment and it is not in any prior document.

The repository contains exactly one commit — `95b943b chore: establish Equina mobile baseline`, dated 2026-06-01, tracking 28 files. Everything produced from Sprint 2 onward is **untracked or uncommitted**:

- the entire `supabase/` tree — **85 files**, including all 34 migrations and all 29 Edge Functions — is untracked;
- `src/backend/` (14 repository and session modules), `src/features/` (all six feature domains), `src/config/`, `src/product/`, and most of `src/ui/` are untracked;
- 21 of 23 documents in `docs/` are untracked;
- `scripts/` (the audit and secret-scan guardrails) is untracked;
- `src/App.tsx`, `src/seed/seed-data.ts`, `src/ui/theme/theme.ts`, and `tests/core-flows.test.ts` are modified and uncommitted.

Nothing in `.gitignore` excludes `supabase/`. The files were simply never added.

Impact. The deployed database schema, every security policy, the media boundary, and the entire Edge Function surface exist in exactly one place: this laptop's filesystem, plus a partial reflection in the linked Supabase project. There is no rollback baseline, no diff history for security-relevant changes, no review trail, and no recovery path if the disk fails or the directory is deleted.

`docs/IOS_RELEASE_AUDIT.md` flagged "no stable rollback baseline" on 2026-07-21; the situation has not improved and has materially worsened because far more work now sits outside version control.

Severity: Critical. Likelihood of eventual loss without action: High. Mitigation cost: under one hour.

This must be resolved before any other item in this assessment. Commit the working tree (secrets excluded — `.env` and `.env.*` are already ignored, and `npm run security:bundle-scan` should be run first), push to a private remote, and only then continue.

4 User Impact

4.1 Who Is Affected

ACTOR	PRESENT EXPOSURE	EXPOSURE AT STAGED ROLLOUT
Rider (primary)	1 internal user with real Horse/Records persistence; everyone else sees labelled sample data	Daily journal, horse records, private documents, AI assistance
Coach	None; no coach role is activated	Rider collaboration, Q&A, annotations
Buyer	Read-only catalog browse	Payment, address, shipping, dispute exposure
Seller	Read-only sample dashboard	Identity/KYC, payout account, tax status
Moderator	Role has backend support but no staffed process	Reviews reported content under deadline
Support	No defined process or owner	Handles disputes, deletion appeals, safety escalations

4.2 Positive Impact

The wedge is sound and the foundations behind it are real. A rider who logs sessions and keeps horse records in Equina gets a durable, private, single place for information currently scattered across paper folders, phone photos, and WhatsApp threads. The Horse/Records vertical is the one journey where implementation, deployment, configuration, and internal rollout have all converged — private buckets, signed upload tickets, ownership RLS, rollback states, and cleanup workers are genuinely in place.

The product's honesty discipline is itself a user-impact asset. The action registry, the production claim registry, and the feature-flag policy prevent the most common harm in early products: a user trusting a fake success state with real data. No CTA currently claims to upload, post, buy, or list when it only mutates demo state.

4.3 Negative Impact And Harm Analysis

Data loss through prototype state (Medium, active today). Large parts of the experience — ride timer and recap, Academy progress, nutrition check-offs, saved shop items, Club counters — are React state that resets on reload. This is correctly labelled, but a rider who logs a ride and closes the app loses it. This is an accepted Sprint 1 boundary, not a defect, and it is the main reason the current build cannot be positioned as a daily-use tool for anyone outside the internal rollout.

Authentication delivery failure (High, blocks rollout). No custom SMTP is configured. Supabase's default mailer is non-production and restricts recipients. A user who signs up outside the allowed set receives nothing, cannot verify, and cannot recover their password. This makes the account journey unusable for real users — the single largest gate on any expansion.

Profile over-exposure (High, latent). Every authenticated account can currently read all profiles, **including location**. For an equestrian product this is not abstract: profile location plus horse identity plus a stable routine describes where a valuable animal is kept and when it is unattended. This must be replaced with a public profile projection and explicit visibility choices before any social surface is enabled worldwide.

Image metadata leakage (High, latent). EXIF removal and image normalization are not implemented. Photos of a horse taken at a stable carry GPS coordinates. Public Club and listing media publishing is correctly disabled for exactly this reason; the gate must hold until the

processing worker described in `docs/EQUINA_MEDIA_PROCESSING_BOUNDARY.md` exists and is proven with before/after fixtures.

Unmoderated social harm (Medium, gated). Club posts, comments, reactions, reports, moderation jobs, and RLS all exist, but no moderation provider is configured and no human moderation process is staffed. Enabling `clubPublishing` without both would expose users to unreviewed content with no functioning report path.

Financial harm (High, correctly gated). Checkout is disabled in both client and database layers, with zero supported buyer and seller countries. The state-machine and webhook code is strong, but Stripe Connect, tax, shipping, legal terms, and dispute operations are absent. Enabling payments in this state would expose buyers to unrecoverable loss and Equina to regulatory liability.

Safety miscommunication (Medium, controlled). Equina positions AI as an assistant, not a coach or diagnostician, with confidence derived from structured context coverage rather than model prose. Health records route to professional escalation. This is the correct posture; the risk is drift, not the current design. `coachChat` must stay off until the provider boundary, consent enforcement, timeout handling, and cross-user isolation tests pass.

5 Data Protection Impact

Equina processes personal data of EU residents on infrastructure in Frankfurt. This section is a DPIA precursor: it identifies the processing, the risk, and the gaps. It is not a completed DPIA and does not constitute legal advice — a qualified reviewer must sign the final version before public launch.

5.1 Categories Of Data Processed

CATEGORY	EXAMPLES	SENSITIVITY	CURRENT CONTROL
Identity	email, display name, avatar	Standard	Supabase Auth; avatar path is server-owned
Location	profile location field, image GPS in EXIF	Elevated	Readable by all authenticated users; EXIF not stripped
Animal records	passport, vet, lab, vaccination, dental, farrier, nutrition	Commercially sensitive; not personal health data	Owner/editor RLS, private buckets, signed URLs
Private documents	uploaded PDFs and images	Elevated	Private Storage, upload tickets, magic-byte checks, scanner adapter
Private messages	marketplace conversations	Elevated	Participant RLS, TLS; server-readable for moderation
AI conversation	Ralf prompts and context	Elevated	Persisted with consent controls; provider not configured
Device tokens	push registration	Standard	Server-only table, not readable via the mobile Data API
Commercial	orders, payouts, disputes, KYC	Elevated	Not active

Horse health data is not special-category personal data under GDPR — it concerns an animal. It is nonetheless treated at an elevated tier because it is commercially valuable, reveals owner behaviour patterns, and users reasonably expect it to be private.

5.2 Processors And Transfers

PROCESSOR	ROLE	REGION	STATUS
Supabase	Database, Auth, Storage, Edge	Frankfurt (EU)	Active
Vercel	Web client hosting	Edge network	Active
Expo / EAS	Build, push delivery	US	Build only; push inactive
SMTP provider	Transactional email	TBD	Not selected
AI provider	Ralf inference	TBD	Not selected
Malware scanner	Upload verification	TBD	Not selected
Stripe	Payments, Connect, KYC	EU/US	Not active
Sentry	Error reporting	TBD	Boundary exists, project not provisioned

Four processors that will handle personal data are unselected. Each requires a data processing agreement, a transfer basis where non-EU, and an entry in the record of processing activities before it may receive production traffic. Selecting an AI provider in particular requires an explicit decision on training-data usage — rider prompts and horse context must not become provider training data.

5.3 Data Subject Rights

RIGHT	IMPLEMENTATION	GAP
Access / portability	request-data-export exists	Synchronous, unpaginated, incomplete — omits parts of Club, media metadata, and seller/listing/order/dispute/review/report history. Untested above 1,000 rows per domain
Erasure	Scheduled deletion with 14-day grace, audit trail, recent-auth requirement	Worker implemented but not deployed ; direct Data API mutation is correctly revoked
Rectification	Profile and preference editing	Rollout-gated
Objection / restriction	Ralf context controls, reduced personalization, history clearing	Rollout-gated

The export gap is the material one. An export that silently omits domains is worse than no export, because it presents itself as complete. Replace it with an asynchronous, paginated, versioned, privately-stored job before `accountSettings` is enabled for anyone beyond the internal user.

5.4 Retention

No retention schedule is defined for horse records, private documents, message bodies, AI conversation history, moderation reports, or operational logs. Deletion is implemented as an account-level operation, not as per-category expiry. A retention schedule is required before public launch and should be produced alongside the privacy policy.

6 Security Impact

6.1 Strengths

Database security is the strongest area of the system, at roughly 92% readiness. Ownership RLS, named-user overrides, private buckets, revoked media-registration grants, worker claim tokens, and server-owned columns are all in place and proven by an isolated Postgres suite that tests cross-user horse isolation, viewer/editor rules, blocked direct media registration, worker concurrency, stale-lease recovery, and stale-token rejection.

The P0 media boundary fix in migration `202607290001` is significant: authenticated clients could previously bypass upload tickets, MIME checks, size checks, content hashes, and moderation queues by writing directly to Storage. That path is now closed and remote negative tests confirm anonymous direct upload, direct media registration, and private signed URL creation are all denied.

`npm run backend:audit` now fails the build on nine classes of regression, and `npm run security:bundle-scan` confirms no server secret reaches the client bundle. These guardrails are the reason this posture is likely to hold.

6.2 Open Security Risks

RISK	SEVERITY	STATE
Exposed Supabase secret key pasted into a prior development conversation	Critical	Not rotated. A secret key bypasses RLS entirely
No staging environment; one project serves both roles	High	Not provisioned
No CAPTCHA/Turnstile, no leaked-password protection, no auth rate-limit monitoring	High	Not configured
Malware scanner not configured	High	Adapter boundary exists, provider absent
No configured Sentry project, dashboard, or alert routing	High	Code boundary exists only
26 transitive dependency findings (18 high, 7 moderate, 1 low) via Expo/RN build tooling	Medium	Tracked; <code>npm audit fix --force</code> must not be run
Physical iOS/Android session-restore proof absent	Medium	Failure-injection test passes in isolation

The exposed secret is the second-most-urgent item in this document after version control. It is not present in tracked files, but it must be treated as compromised: rotate or revoke it in the Supabase dashboard and review the access logs.

7 Operational Impact

7.1 What Running Equina Requires

Activating the currently-gated capabilities does not just add code paths; it creates standing operational obligations that no one has been assigned:

OBLIGATION	TRIGGERED BY	OWNER
Content moderation queue with review deadlines	<code>clubPublishing</code> , <code>clubInteractions</code>	Unassigned

OBLIGATION	TRIGGERED BY	OWNER
Abuse report triage and sanction decisions	any social surface	Unassigned
Dispute adjudication and refund decisions	shopTransactions	Unassigned
Deletion and export request handling	accountSettings	Unassigned
AI safety escalation review	coachChat	Unassigned
Push delivery and invalid-token hygiene	pushNotifications	Unassigned
On-call for 5 schedulers and 29 functions	any expansion	Unassigned

This is the least-documented dimension of the project. Substantial backend machinery exists for moderation, reporting, sanctions, disputes, and deletion, but there is no operating model behind it — no staffing, no response-time commitment, no escalation path, no queue ownership. A moderation queue with no moderator is not a safety control.

7.2 Observability

Request IDs, stable redacted errors, and an optional server-only Sentry boundary exist. There is no configured project, no dashboard, and no alert routing. The audit lists eight required alert classes — auth delivery failures, Edge 4xx/5xx and latency, moderation backlog, upload ticket mismatch and cleanup backlog, notification dead letters, deletion/export failures, Stripe webhook drift, and unresolved Ralf safety events. None are live. Today, a failure in the one active scheduler would be discovered by manual inspection.

7.3 Deployment And Environments

One environment serves as both staging and production. Promotion must be migration-by-migration and function-by-function, but there is nowhere to promote *from*. Additionally, the EAS remote environment could not be inspected because the local CLI has no authenticated Expo account — local `eas.json` and `app.json` are clean, but remote EAS variables remain unverified.

8 Business And Commercial Impact

8.1 Model Alignment

The blueprint ranks subscription first (`Equina Plus` , `Coach Pro`), resale fees second, education third, and listing fees last. The engineering reality matches this ranking, which is a good sign: the subscription-supporting verticals (Horse, Records, Academy, Ralf) are further along than the commerce verticals. Nothing in the current state forces a commerce-first launch.

8.2 Revenue Timing

No revenue is possible in the current state. Subscription requires `accountSettings` proven end-to-end; resale requires the entire checkout program. The realistic sequence is:

1. Prove the daily loop with internal and then invited users — no revenue, high learning value.
2. Enable subscription once accounts, export, and deletion are proven — first revenue.
3. Treat commerce as a separate regulated launch — later, and only with legal and finance sign-off.

Attempting to shortcut to step 3 is the principal commercial risk, because it converts a product problem into a regulatory one.

8.3 Regulatory Exposure At Commerce Launch

`docs/EQUINA_CHECKOUT_ACTIVATION_CHECKLIST.md` is thorough and its gates should be treated as binding. The exposures it correctly identifies: Stripe Connect account model and cross-border fund flows, per-country KYC, VAT/OSS and deemed-supplier analysis, marketplace seller reporting, consumer cancellation rights, prohibited-goods policy (damaged helmets, unsafe tack, counterfeits, recalled products), shipping insurance and loss ownership, and dispute legal review. To this list add EU marketplace transparency obligations and platform content-moderation duties, which attach to the social surfaces independently of whether payments are enabled.

The checklist's closing rule is the right one and worth restating: **country expansion is a new approval, not a percentage rollout.**

9 Consolidated Risk Register

Severity reflects impact if unmitigated. Priority reflects what to do first.

#	RISK	SEVERITY	LIKELIHOOD	MITIGATION	PRIORITY
R1	Backend and docs exist only on one disk; no version-control baseline	Critical	High	Commit and push the full tree to a private remote today	P0
R2	Supabase secret key treated as compromised, not rotated	Critical	Medium	Rotate/revoke, review access logs, verify only publishable keys ship	P0
R3	No production email delivery; users cannot verify or recover	High	Certain at rollout	Custom SMTP, SPF/DKIM/DMARC, templates, delivery monitoring	P0
R4	No staging environment	High	High	Provision a second Supabase project and a CI promotion path	P0
R5	Activated vertical unproven on hosted accounts	High	Medium	Hosted Horse→Record→upload→restart plus Rider A/B/C isolation	P0
R6	Profile location readable by every authenticated account	High	Certain at social rollout	Public profile projection plus explicit visibility choices	P1
R7	EXIF/GPS not stripped from uploaded images	High	Certain at media rollout	Build the processing worker; keep public media disabled	P1
R8	Data export incomplete and unpaginated	High	Certain at account rollout	Async, paginated, versioned, privately-stored export job	P1
R9	No moderation, dispute, or support operating model	High	Certain at social/commerce rollout	Define owners, SLAs, escalation before enabling any flag	P1
R10	No alerting or dashboards	High	High	Provision Sentry, add the eight alert classes	P1
R11	Malware scanner unconfigured	High	Medium	Contract provider, prove clean/EICAR/timeout/5xx cases	P1
R12	4 of 5 workers implemented but undeployed	Medium	Medium	Deploy with secrets and schedules, one worker at a time	P1

#	RISK	SEVERITY	LIKELIHOOD	MITIGATION	PRIORITY
R13	Client still uses mock surfaces for Club, seller drafts, checkout	Medium	Certain	Wire one vertical at a time in the documented order	P1
R14	src/App.tsx is 15,971 lines / 76 components / 76 useState hooks	Medium	High	Decompose by route; see the technical design document	P2
R15	26 transitive dependency findings	Medium	Medium	Track Expo-compatible upgrades; never <code>audit fix --force</code>	P2
R16	Physical device session-restore unproven	Medium	Medium	iOS and Android process-kill matrix	P2
R17	Apple/Google providers unconfigured	Medium	Certain at rollout	Full device matrix before exposing buttons	P2
R18	No retention schedule for any data category	Medium	Certain at launch	Define per-category retention with the privacy policy	P2
R19	Commerce regulatory program not started	High	Deferred	Keep checkout disabled; treat as separate launch	P3

10 Impact Of Doing Nothing

If the system is left as it is:

- The version-control risk compounds daily. Every additional hour of work increases what a disk failure destroys.
- The exposed secret remains valid and RLS-bypassing for as long as it is not rotated.
- No user harm accrues, because the fail-closed posture holds. The product simply stops progressing.

The asymmetry matters: the product-side cost of pausing is low, but the two P0 infrastructure risks are time-sensitive and cheap to fix. R1 and R2 should be closed regardless of whether any further product work is scheduled.

11 Recommendation

Do not expand any capability flag until R1–R5 are closed. They are ordered, cheap relative to their severity, and every subsequent decision depends on them.

Recommended sequence:

1. **Today** — commit and push the working tree to a private remote (R1); rotate the exposed Supabase secret (R2).
2. **Week 1** — configure custom SMTP with domain authentication, CAPTCHA, and leaked-password protection (R3); provision a staging Supabase project with a CI promotion path (R4).
3. **Week 2** — run the hosted Horse→Record→upload→restart journey and the Rider A/B/C isolation matrix on real accounts; run physical iOS and Android session-restore (R5, R16).
4. **Week 3–4** — provision Sentry and the eight alert classes (R10); replace the export with an async paginated job (R8); ship the public profile projection (R6).

5. **Then, and only then** — expand the internal Horse/Records override beyond one user, and begin the media processing worker (R7) that unblocks Club.
6. **Separately, not on this track** — start the commerce regulatory program (R19) with named legal, finance, and support owners.

Two structural recommendations sit alongside the sequence. First, assign an owner to every operational obligation in section 7.1 before enabling the flag that creates it; a queue without a human is not a control. Second, keep the existing honesty discipline — the action registry, the production claim registry, and the release gate are the reason this system carries so little user-facing risk despite being far from finished. They should survive contact with launch pressure.

12 Sign-Off

ROLE	DECISION REQUIRED	STATUS
Engineering	R1, R2, R4, R5 execution	Open
Product	Rollout sequence and internal cohort size	Open
Privacy/Legal	Processor selection, DPIA completion, retention schedule	Open
Finance/Legal	Commerce program go/no-go	Deferred
Operations	Owners and SLAs for section 7.1	Open

Related documents: [EQUINA_TECHNICAL_DESIGN.md](#) , [EQUINA_SERVICE_DESIGN.md](#) , [EQUINA_BACKEND_AUDIT_2026-07-29.md](#) , [EQUINA_BACKEND_DEPLOYMENT_INVENTORY.md](#) , [EQUINA_CHECKOUT_ACTIVATION_CHECKLIST.md](#) , [EQUINA_FEATURE_FLAG_POLICY.md](#) , [EQUINA_ACTION_REGISTRY.md](#) .

Technical Design

The system as it is actually built, the two structural problems in it, and the target architecture.

DESIGN DATE	2026-08-25
APPLIES TO	Equina 2.0 - copie working tree, Supabase project vdcrllyzbdjonotrujb (Frankfurt)
VERIFIED ON 2026-08-25	npm test and npm run typecheck both pass.

This document describes the system as it is actually built, names the two structural problems in it, and defines the target architecture. It is a design record, not a tutorial — it assumes the reader has `docs/EQUINA_BACKEND_AUDIT_2026-07-29.md` and `docs/EQUINA_BACKEND_DEPLOYMENT_INVENTORY.md` available.

1 Goals And Constraints

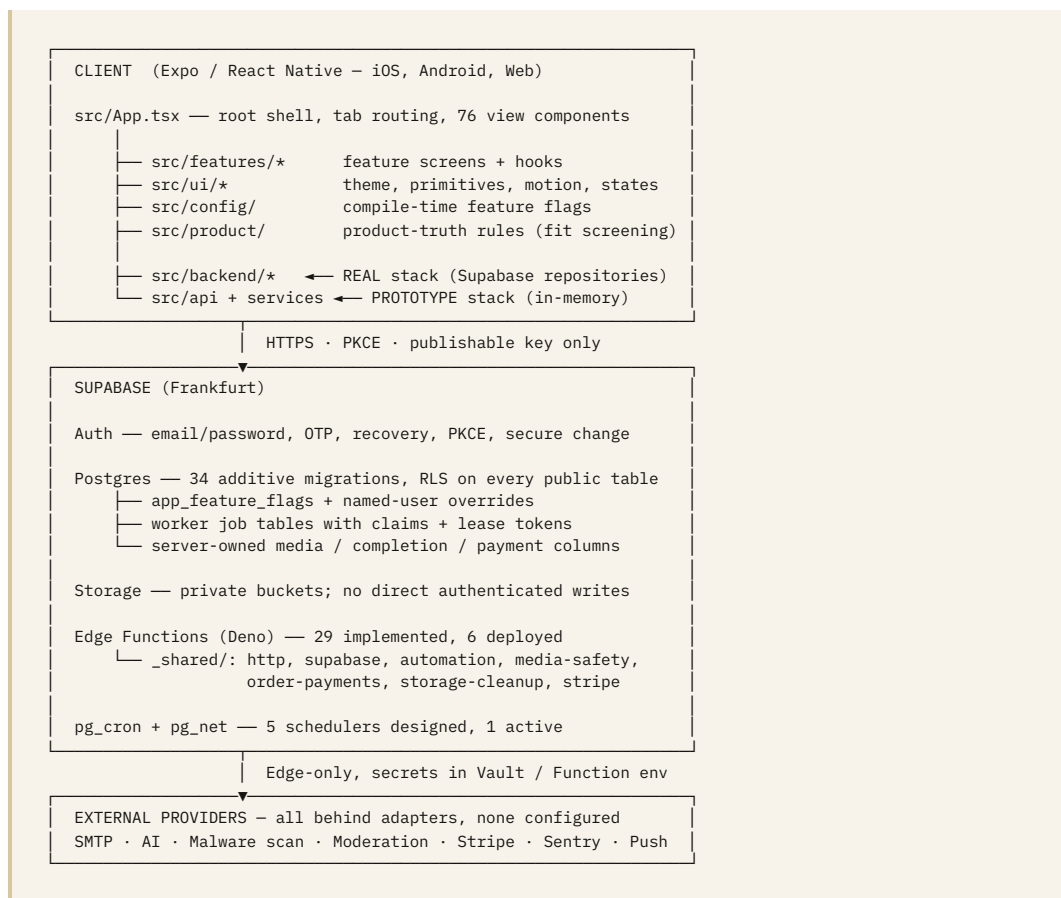
Design Goals

1. **Fail closed.** A capability is unavailable unless every layer independently confirms it is safe. No single switch can turn on a product surface.
2. **No fake success.** The client may show a read-only route for a disabled mutation, but never a success state for something that did not persist.
3. **Server-owned truth.** Security-relevant fields — media paths, completion timestamps, moderation status, payment state — are written by the server, never by the client.
4. **Additive migrations only.** An applied migration is never edited or removed; corrections ship forward.
5. **Provider-agnostic boundaries.** AI, malware scanning, payments, email, and moderation sit behind adapters so a provider can be swapped without touching product code.

Constraints

- Single codebase targeting iOS, Android, and web via Expo SDK 55 / React Native 0.83 / React 19.
- EU data residency (Frankfurt).
- One hosted environment today; staging is required but not provisioned.
- Client bundles may contain publishable keys only — enforced by `npm run security:bundle-scan`.

2 System Overview



3 The Three-Layer Activation Gate

This is the most important design decision in the system and the reason its risk posture is defensible. A capability is live only when:

```
compile-time failsafe AND authenticated backend capability AND screen precondition
```

Layer 1 — Compile-time failsafe

src/config/feature-flags.ts reads EXPO_PUBLIC_ENABLE_* environment variables at build time. Three flags are hardcoded false in source (clubPublishing, clubInteractions, shopTransactions, shopListingCreation) and cannot be enabled by configuration alone. Each flag carries a featureFlagReason string used verbatim in disabled-state copy, so the UI explains the boundary instead of hiding it.

This layer is an emergency off switch. It exists so a bad database rollout cannot activate a surface in a build that was never tested for it.

Layer 2 — Authenticated backend capability

supabase/functions/backend-capabilities/index.ts returns a per-session capability object. Its design has two properties worth preserving:

Per-session flag resolution. The function attempts `requireUser(request)` , then resolves `featureFlagsForSession(token)` — which applies global flags, rollout percentage, and named-user overrides. A 401 is caught and downgraded to an empty flag set rather than an error, so the unauthenticated response is a valid fail-closed object rather than a failure.

Fail-closed provider conjunction. A database flag is never sufficient on its own. Each capability ANDs the flag with a check that the provider secrets actually exist in the Edge environment:

CAPABILITY	DATABASE FLAG	AND PROVIDER CHECK
accountSettings	account_settings	ACCOUNT_AUTOMATION_SECRET
coachChat	coach_chat	AI URL + key + model, or an explicit localhost-only dev fallback
pushNotifications	push_notifications	NOTIFICATION_AUTOMATION_SECRET
clubPublishing	club_publishing	moderation URL + token + automation secret
listingCreation	shop_listing_creation	moderation URL + token + automation secret
checkout	shop_transactions	Stripe key + webhook secret + terms version + Connect return/refresh URLs
records / horseManagement / clubInteractions / messaging	flag only	—

The consequence: turning on a database flag for a capability whose provider is unconfigured changes nothing. This is why the current remote response returns `auth: true` and `ten false` values despite substantial deployed code.

The dev fallback for `coachChat` is deliberately constrained to `SUPABASE_URL` containing `127.0.0.1` or `localhost` , so it cannot activate against a hosted project.

Layer 3 — Screen precondition

The screen itself checks its own requirements at action time: an active session, a selected horse, a non-empty prompt, an active participant relationship, a granted OS permission. This is where `docs/EQUINA_EXPERIENCE_STATE_CONTRACT.md` applies — loading, empty, error, offline, and permission-denied must all be implemented before the flag may move.

4 Client Architecture

4.1 Layers

LAYER	LOCATION	RESPONSIBILITY
Shell	src/App.tsx	Tab routing, screen composition, prototype state
Features	src/features/{account,coach,messaging,notifications,onboarding,records,ride}	Screens plus their controller hooks
UI system	src/ui/{theme,primitives,motion,layout,states,conversation,settings}	Tokens, primitives, haptics, reduced-motion, state copy
Product rules	src/product/product-truth.ts	Truth-constrained derivations (qualitative fit screening)

LAYER	LOCATION	RESPONSIBILITY
Backend	<code>src/backend/*</code>	Supabase repositories, edge client, session storage
Prototype	<code>src/api/</code> , <code>src/services/</code> , <code>src/seed/</code>	In-memory store used for seeded demo surfaces
Domain	<code>src/domain/types.ts</code>	Shared entity types

Feature controllers follow a consistent shape: `useAccount` , `useHorseRecords` , `useCoachConversation` , `useShopConversation` , `useEquinaSession` , `usePushRegistration` . Each owns loading/error/optimistic state for its domain and calls the matching repository. This is the healthy part of the client and the pattern the rest should converge on.

4.2 Backend Client Layer

`EquinaBackend` (`src/backend/equina-backend.ts`) is a lazily-constructed singleton composing seven repositories plus an `EdgeClient` over one Supabase client. Its `connect()` resolves session and capabilities in parallel — one round trip to establish both identity and what the app is allowed to do.

`getSupabaseClient()` configures PKCE flow, `processLock` for concurrent refresh safety, `detectSessionInUrl: false` , and platform-conditional storage. On native it binds an `AppState` listener that starts and stops auto-refresh with foreground state, avoiding background token churn.

`EdgeClient.invoke()` attaches a per-call `x-request-id` UUID, giving every Edge invocation a correlation ID that survives into redacted server logs.

`EquinaBackendError` carries `code` and a `retryable` flag derived from HTTP status — status 0 or ≥ 500 is retryable, 4xx is not. This is what lets controllers distinguish "retry this" from "fix your input" without parsing prose.

4.3 Native Session Storage

Supabase sessions exceed `expo-secure-store` 's per-item limit, so `chunked-session-storage.ts` splits them across keys with a manifest. The design is generation-safe:

1. Write every chunk of a **new generation** under new keys.
2. Atomically swap the manifest to point at the new generation.
3. Only then remove the previous generation.

Per-key mutations are serialized, legacy manifests migrate in place, and a failure-injection test proves a partial write preserves the previous session without leaking new chunks. Keychain accessibility is `WHEN_UNLOCKED_THIS_DEVICE_ONLY` — sessions do not migrate to a new device via backup.

Open gate: physical iOS and Android process-kill proof has not been run. The isolated test proves the algorithm; it does not prove Keychain behaviour under real app termination.

5 Backend Architecture

5.1 Database

34 additive migrations spanning `202607210001_core` through `202607290005_data_export_cleanup_leases`, organized by domain: core, records, club, marketplace, storage/realtime security, operations, safety, collaboration, rollout/checkout hardening, moderation, notifications, account deletion, onboarding, media boundary, internal rollout, and worker leases.

Every public table has RLS and a primary key. `npm run backend:audit` fails the build on nine regression classes:

- public tables without RLS or primary keys;
- anonymous mutation grants;
- authenticated media registration grants;
- direct authenticated Storage write policies;
- client access to server-owned media/completion columns;
- unsafe `security definer` search paths;
- unexpected anonymous `security definer` execution;
- unindexed foreign keys.

This guardrail is the reason the security posture is likely to survive future changes. It should be wired into CI as a required check the moment CI exists.

5.2 The Media Boundary

The most consequential security design in the system. Migration `202607290001` closed a P0 where authenticated clients could write directly to Storage and insert media registration rows, bypassing every check.

The only valid path is now:

```
create-upload-ticket → client receives a signed, scoped, expiring upload target
|
client uploads directly to private Storage
|
complete-upload → verifies owner/editor, ticket expiry, bucket/path ownership,
                  declared size, declared MIME, magic bytes, SHA-256 content hash
|
malware scanner adapter (short-lived signed URL, MIME, size, hash)
|
verdict "clean" → create active media reference
verdict "malicious" → reject, queue for private Storage cleanup
```

Direct writes to `horse_record_files`, `club_post_media`, `listing_photos`, and `dispute_evidence` are revoked. Profile avatar, onboarding completion, and horse photo paths are server-owned, and the onboarding RPC cannot set an arbitrary horse photo path.

`MALWARE_SCAN_REQUIRED=true` makes provider failure fail closed. The scanner is not yet contracted.

Known gap: EXIF stripping, image normalization, and video transcoding are not implemented. The required processor contract is specified in `docs/EQUINA_MEDIA_PROCESSING_BOUNDARY.md` — claim with lease token, download via service signed URL, decode and re-encode, strip all metadata, enforce dimensions, recompute MIME/size/hash, write to a new immutable path, atomically

replace the active reference, queue the source for cleanup, complete under the same lease token. Until it exists, public media publishing stays disabled and image upload is approved for named internal users only.

5.3 Edge Functions

29 implemented, 6 deployed. `supabase/functions/_shared/` provides the common surface: `http.ts` (request IDs, typed `HttpError`, CORS preflight, method enforcement, redacted error responses), `supabase.ts` (`requireUser`, `featureFlagsForSession`), `automation.ts` (worker secret verification), `media-safety.ts`, `order-payments.ts`, `storage-cleanup.ts`, `stripe.ts`.

Deployed today:

FUNCTION	VER	JWT	PURPOSE
backend-capabilities	6	no	Per-session capability resolution
create-upload-ticket	2	yes	Issue scoped signed upload target
complete-upload	3	yes	Verify and register media
delete-upload-asset	2	yes	Remove media through the boundary
delete-horse-record	2	yes	Server-authorized record deletion
process-storage-cleanup	2	no	Scheduled orphan cleanup

Undeployed and remote-disabled: all Club, messaging, notification, account deletion/export, Ralf, moderation, seller, tax, Stripe, dispute, payout, and order-worker functions.

Worker secret comparison is centralized through SHA-256 digest comparison, and moderation and Storage cleanup now hold **separate** secrets rather than reusing the order automation credential. This blast-radius separation is correct; the corresponding secrets must exist before those functions are deployed.

5.4 Async Work Model

Five workers share one durable pattern:

1. **Atomic claim** — a database-level claim prevents two workers taking the same job.
2. **Lease token** — the worker holds a token; a stale worker whose lease expired cannot publish its result.
3. **Attempt counting and bounded retry** — failures are counted, not retried forever.
4. **Stale recovery** — expired leases return jobs to the queue.
5. **Dead-letter** — exhausted jobs are preserved, not dropped.

Proven in the isolated Postgres suite for worker concurrency, stale-lease recovery, and stale-token rejection.

WORKER	IMPLEMENTED	DEPLOYED	SCHEDULED
Storage cleanup	yes	yes	/5 *
Content moderation	yes	no	no
Notification outbox	yes	no	no

WORKER	IMPLEMENTED	DEPLOYED	SCHEDULED
Account deletion	yes	no	no
Data export cleanup	yes	no	no

Two designs still need changing before their capabilities activate: `request-data-export` is **synchronous and unpaginated**, and order deadline processing must be redesigned around durable, idempotent provider operations before checkout can be considered.

5.5 Secret Isolation

SECRET CLASS	LOCATION	NEVER IN
Publishable Supabase key	Expo/Vercel client env	—
Supabase secret key	Supabase dashboard only	Client, docs, chat, git
Worker automation secrets	Edge Function env + Vault	Client, docs, git
Provider secrets (AI, scanner, Stripe, SMTP)	Edge Function env / provider console	EXPO_PUBLIC_*, logs, docs
Apple/Google client secrets	Provider console + Supabase Auth	EXPO_PUBLIC_*

`npm run security:bundle-scan` inspects exported web artifacts for server-secret markers and currently reports zero. **Outstanding:** one Supabase secret key was pasted into a prior development conversation and must be treated as compromised until rotated.

6 Structural Problem 1: The Dual Data Stack

Two complete data stacks coexist inside the same running component.

`src/App.tsx:901` calls `createSeededEquinaApi()`, which builds `createEquinaApi()` from `src/api/equina-api.ts` — an in-memory `EquinaStore` with `AuthService`, `ProfileService`, `ListingService`, `OrderService`, `CommunityService`, and a `DeterministicAiProvider`. The same file imports `EquinaBackend` for the real Supabase path.

The result is a screen-by-screen split with no enforced boundary:

SURFACE	STACK
Account, Auth, Coach, Records, Horse, Shop messaging	Real (<code>src/backend/*</code>)
Home, Ride, Academy, Club, Shop browse/checkout/orders/saved, Seller	Prototype (in-memory seed)

This is a reasonable transitional state — it is how the product stayed demonstrable while the backend was built — but it carries three real costs. It is the direct cause of the "reload resets" behaviour catalogued in the action registry. It means `tests/core-flows.test.ts` exercises the prototype stack, so passing tests say less about production behaviour than they appear to. And it makes every future activation a migration rather than a flag change, because the screen must be rewritten, not just unlocked.

Target. Each surface migrates to a feature controller hook over a repository, matching `useHorseRecords` / `useAccount`. The prototype stack shrinks to seeded fixtures for tests and screenshots only, and `src/api/equina-api.ts` plus `src/services/*` are deleted once no screen

imports them. The migration order from the backend audit still holds: Horse CRUD → Records without files → private record files → Club text → Club media → marketplace browse → listing drafts → messaging → checkout.

7 Structural Problem 2: The App.tsx Monolith

`src/App.tsx` is **15,971 lines** — 54% of the 29,567-line `src` tree. It contains **76 component functions**, **76 `useState` calls**, and a single `StyleSheet.create` block starting at line 6,902 that runs for roughly 9,000 lines.

Concretely this means: every screen's state lives in one component's scope; a change to a Shop card can trigger a re-render decision affecting Home; the style sheet cannot be reasoned about per screen; two people cannot work on different tabs without conflicting; and the file is large enough to degrade editor and type-checker responsiveness.

The `src/features/` and `src/ui/` directories show the intended structure — `OnboardingScreen`, `AccountScreen`, `RideExperience`, `CoachScreen`, `HorseRecordSheets`, `ShopConversationScreen` have already been extracted. The work is half done and stalled.

Target decomposition:

```
src/app/           root shell, providers, tab router (< 400 lines)
src/features/home/  HomeScreen + useHome
src/features/stable/ StableScreen + sections
src/features/academy/ AcademyScreen, library, video, lesson
src/features/club/  CommunityScreen + useClub
src/features/shop/  buyer + seller routes + useShop
src/ui/styles/      per-feature style modules
```

Every remaining component in `App.tsx` moves next to the feature it serves, with its styles. This is mechanical, low-risk, and should be done **incrementally alongside** each backend-wiring task — extracting a screen at the moment it migrates from prototype to repository, rather than as a separate refactor sprint.

Prerequisite: this work must not begin until the repository is under version control (see `EQUINA_IMPACT_ASSESSMENT.md` §3). A 16,000-line file split with one commit as the only baseline is unrecoverable if it goes wrong.

8 Environments And Deployment

ENVIRONMENT	SUPABASE	CLIENT FLAGS	DATA	STATUS
Local	Local CLI, or linked project for explicit remote tests	Demo off; only the target capability on	Synthetic only	Working
Staging	Separate project	Internal distribution, named users	Synthetic + consented QA	Not provisioned
Production	Existing project, post-promotion	Default off, gradual overrides	Real users	Serving both roles

Promotion must be migration-by-migration and function-by-function. Database passwords, service keys, automation secrets, provider secrets, and private Storage objects are never copied between environments.

Web client: `https://equina-five.vercel.app` , deployment `dp1_HkDJ3uWi5J3E3JqHPDLx5GDkVxHa` , publishable configuration only, Horse/Records compile switches on, all other privileged switches off, demo fallback off.

Native: EAS remote environment **could not be inspected** — the local CLI has no authenticated Expo account. Local `eas.json` and `app.json` contain no credential values, but remote EAS variables remain an explicit audit blocker.

Emergency Rollback

- 1. Disable the client compile-time switch and clear the named database override.
- 2. Stop the matching cron job or provider traffic.
- 3. Preserve rows and cleanup queues; never edit or remove an applied migration.
- 4. For an upload incident, remove `complete-upload` only after flags are disabled.
- 5. Rotate the affected server secret and inspect redacted logs by request ID.
- 6. Ship an additive forward fix, rerun isolation tests and the bundle scan, then restore one named internal user before any wider rollout.

9 Test And Verification Strategy

LAYER	COMMAND	COVERS
Core flows	<code>npm test → core-flows.test.ts</code>	Prototype-stack product rules, fit screening
Security contracts	<code>npm test → backend-contract.test.ts</code>	Capability shape, repository contracts
Migrations	<code>npm test → backend-migrations.test.ts</code>	Isolated Postgres (PGLite): rollout isolation, cross-user horse isolation, viewer/editor rules, blocked media registration, worker concurrency, stale-lease recovery, stale-token rejection
Types	<code>npm run typecheck</code>	Whole tree
Deno	<code>npm run backend:check</code>	All 29 Edge Functions
Schema guardrail	<code>npm run backend:audit</code>	Nine RLS/grant/index regression classes
Bundle secrets	<code>npm run security:bundle-scan</code>	Exported web artifacts
Visual	<code>tests/visual/*.mjs</code> (Playwright)	Onboarding, sprint smoke, polish audits

What this does not prove. PGLite cannot prove hosted authenticated A/B/C isolation, Auth email delivery, Realtime behaviour, provider integrations, or physical-device behaviour. Those are mandatory staging gates and are listed as such in the backlog. Note also that `core-flows.test.ts` runs against the prototype stack, so its green result is not evidence about production data paths.

Not yet run: real email delivery, authenticated Horse→Record→upload→restart on hosted accounts, Rider A/B/C hosted isolation, physical iOS/Android session restore, two-device Realtime/offline/reconnect, Apple or Google provider sign-in.

10 Observability Design

Implemented: per-request UUIDs from `EdgeClient` through `_shared/http.ts`, stable redacted error responses, typed `HttpError` with status-derived retryability, and an optional **server-only** Sentry boundary.

Not provisioned: Sentry project, dashboards, alert routing. Required alert classes:

1. Auth delivery and callback failures
2. Edge 4xx/5xx rate and latency
3. Moderation backlog and job failures
4. Upload ticket mismatch and cleanup backlog
5. Notification dead letters
6. Account deletion and export failures
7. Stripe webhook retries, payment-state drift, transfer failures
8. Ralf provider errors and unresolved safety events

Privacy constraint: health records, message bodies, and payment state never enter analytics or error payloads. Push previews carry only `New message about [listing]` plus a route and conversation ID.

11 Known Technical Debt

#	ITEM	SEVERITY	NOTES
T1	Repository not under version control	Critical	85 supabase files untracked; see impact assessment §3
T2	Dual data stack	High	§6
T3	<code>App.tsx</code> at 15,971 lines	High	§7
T4	<code>request-data-export</code> synchronous and unpaginated	High	Must become an async paginated job
T5	Broad profile reads including location	High	Needs a public projection or RPC
T6	No EXIF/normalization worker	High	Blocks all public media
T7	Order deadline processing not idempotent-durable	High	Blocks checkout
T8	26 transitive dependency findings (18 high)	Medium	Expo/RN build tooling; never <code>audit fix --force</code>
T9	4 of 5 workers undeployed	Medium	Deploy one at a time with secrets and schedules
T10	No CI	Medium	<code>backend:audit</code> , <code>typecheck</code> , <code>test</code> , <code>bundle-scan</code> should gate merges

12 Decision Log

#	DECISION	RATIONALE	CONSEQUENCE
D1	Supabase over a custom backend	RLS as the primary authorization boundary; managed Auth/Storage/Edge; EU residency	Postgres policy correctness is the security model; <code>backend:audit</code> must guard it
D2	Three-layer activation gate	No single actor can enable a surface	Activation is slower by design; flags alone cannot ship a feature
D3	Capability endpoint ANDs flags with provider-secret presence	Prevents enabling a capability whose provider is absent	Provider config is a hard dependency of rollout
D4	Additive migrations only	Preserves rollback and audit trail	Corrections ship forward; the migration count grows
D5	Ticketed upload with server-side verification	Client cannot bypass MIME/size/hash/scan checks	Two round trips per upload
D6	Claims plus lease tokens for all workers	Stale workers cannot publish	Every worker needs a job table and lease logic
D7	Chunked generation-safe secure session storage	Supabase sessions exceed SecureStore item limits	Custom storage needs device-level proof
D8	Protected payment with delayed transfer, not escrow	Escrow requires a licensed provider	Must never be marketed as escrow
D9	AI as assistant with derived confidence	Model prose is not a confidence signal	Confidence comes from structured context coverage only
D10	Seeded prototype stack retained during backend build	Kept the product demonstrable	Created D-debt T2; must be retired surface by surface

13 Immediate Technical Priorities

1. **Commit and push everything to a private remote.** Nothing else in this document is safe to start first.
2. **Rotate the exposed Supabase secret key** and review access logs.
3. **Provision staging** and a CI promotion path; wire `typecheck`, `test`, `backend:check`, `backend:audit`, and `security:bundle-scan` as required checks.
4. **Configure custom SMTP**, SPF/DKIM/DMARC, templates, CAPTCHA, and leaked-password protection.
5. **Prove the activated vertical on hosted accounts** — Horse→Record→upload→restart, plus Rider A/B/C isolation, plus physical iOS/Android session restore.
6. **Then** begin the dual-stack migration and the `App.tsx` decomposition together, one surface at a time.

Related documents: `EQUINA_IMPACT_ASSESSMENT.md`, `EQUINA_SERVICE_DESIGN.md`, `EQUINA_BACKEND_SECURITY_MODEL.md`, `EQUINA_BACKEND_RUNBOOK.md`, `EQUINA_BACKEND_DEPLOYMENT_INVENTORY.md`, `EQUINA_MEDIA_PROCESSING_BOUNDARY.md`, `EQUINA_FEATURE_FLAG_POLICY.md`.

Service Design

The whole service — the app, the humans behind it, and the moments where it can fail a rider.

DESIGN DATE	2026-08-25
SCOPE	the whole Equina service — the app, the humans behind it, the processes that must run for it to be trustworthy, and the moments where it can fail a rider.

The product documentation in this repository is strong on interface and backend. It is nearly silent on the service: who answers a report, who adjudicates a dispute, who reviews a safety escalation, and how fast. This document closes that gap and defines the service that has to exist before the flags in `docs/EQUINA_FEATURE_FLAG_POLICY.md` can move.

1 Service Proposition

Equina is the place a rider keeps what matters about their horse — training, care, records, and questions — and gets useful, honest help with it.

The service promise has four parts, in priority order:

1. **What you record stays yours and stays safe.** Private by default, exportable, deletable.
2. **What we tell you is honest about its own limits.** Guidance, not diagnosis; screening, not fit certification; confidence derived from data coverage, not from tone.
3. **What you can do, you can actually do.** No control claims an outcome it cannot deliver.
4. **When something goes wrong, a human handles it.** Reports, disputes, deletion requests, and safety escalations reach a person within a stated time.

Promises 1–3 are largely designed and partly built. **Promise 4 has no owner, no process, and no staffing.** That is the central finding of this document.

2 Service Principles

Derived from the product psychology contract and the experience laws, restated as service rules:

PRINCIPLE	WHAT IT FORBIDS	WHAT IT REQUIRES
Honest capability	Fake success states, invented percentages, "escrow" language, unsupported health claims	Every disabled action states its boundary plainly
Named source	Presenting demo quantities as a feeding plan; treating model prose as confidence	Every visible metric names its source or is removed
Healthy repetition	Streak anxiety, fake urgency, random rewards, notification bait	Competence, autonomy, relatedness, and trust in every loop
Human recourse	Automated-only moderation, disputes, or deletions	A named owner and a response time for every queue
Professional escalation	Diagnosing, prescribing, confirming saddle fit	Routing to vet, saddler, or coach with the triggering observations shown
Reversibility	Silent destructive actions	Edit, undo, exit, grace periods, and confirmations

3 Actors

3.1 Users

ACTOR	CORE NEED	SERVED TODAY
Rider (primary)	One trusted place for training and horse records	Partly — 1 internal user has real persistence; everyone else sees labelled samples
Coach	Follow riders, answer questions, annotate	No — role exists in the type system, no journey
Buyer	Inspect gear honestly before committing	Read-only browse
Seller	List gear, get paid, be trusted	Read-only sample dashboard

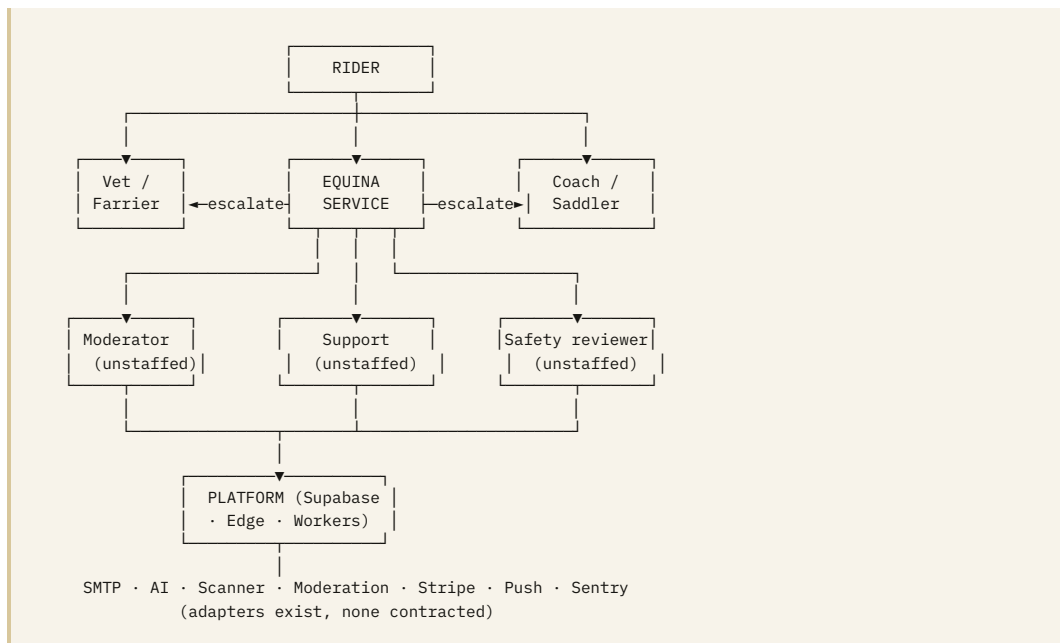
3.2 Service Providers (internal)

ACTOR	RESPONSIBILITY	STATUS
Moderator	Review reported content within deadline, decide sanctions	Backend exists (<code>moderate-report</code> , <code>lift-sanction</code> , moderation jobs, RLS). No person, no process, no SLA
Support	Handle disputes, deletion appeals, delivery failures, account problems	Does not exist
Safety reviewer	Review Ralf escalations and health-flag events	Does not exist
On-call engineer	Respond to scheduler, function, and provider failures	Does not exist ; no alerting to respond to
Finance/legal	Payout reconciliation, tax, terms, prohibited goods	Not engaged

3.3 External Providers

SMTP, AI, malware scanning, content moderation, Stripe, Expo push, Sentry. All seven sit behind adapters. **None is contracted or configured.** Each is a service dependency, not just a technical one: an unconfigured SMTP provider does not degrade the service, it removes the ability to create an account at all.

4 Service Ecosystem Map



Three of the four internal service roles are empty boxes. The platform is the most complete part of the ecosystem and the humans are the least.

5 Service Blueprints

Each blueprint uses five layers: **Evidence** (what the rider sees), **Rider actions**, **Frontstage** (system response the rider perceives), **Backstage** (system work the rider does not see), and **Support process** (human work). A  marks a gap that blocks activation.

5.1 Onboarding And Account Creation

LAYER	DETAIL
Evidence	Discipline imagery, three-stage progress, email inbox, starter pack
Rider actions	Choose discipline and level → add horse (optional) → preview → enter email → verify → land on Home
Frontstage	Draft answers held locally; social buttons only where the provider is proven; verification screen with resend cooldown; completion transition bridging the discipline image into Home
Backstage	Draft persisted locally until verification; Supabase Auth (PKCE, secure change enabled); <code>complete_equina_onboarding</code> writes profile, preferences, and one primary horse idempotently ; <code>onboarding_starter_packs</code> prevents duplicate rewards on retry
Support	 Nobody handles "I never got the email"

Failure points.  **No custom SMTP** — the default mailer restricts recipients, so verification silently fails for most addresses. This is the single hardest stop in the whole service.  **No CAPTCHA or leaked-password protection.**  Apple/Google buttons must stay hidden until each provider's full device matrix passes.

Design strength worth keeping: rider context is collected *before* account creation, and the handoff is idempotent, so a retry cannot produce a duplicate horse. The service asks for commitment only

after it has shown value.

5.2 Daily Ride Loop

LAYER	DETAIL
Evidence	Home briefing with one suggested ride; three-phase timer; factual recap
Rider actions	Open app → read one suggestion → start ride → advance phases → finish → record how the horse felt
Frontstage	Home is a state machine — first session, pre-ride, active, post-ride, care due, recovery day, no horse — with one primary action per state
Backstage	△ Ride timer, phases, and check-in are React state only
Support	None needed

Failure point. △ **The recap is lost on reload.** This is the retention engine of the entire product and it does not persist. The daily loop cannot be offered to any real user until ride entries are written through a repository.

Design strength: the recap reports elapsed time, completed phases, and a rider-entered horse-feel check-in — all facts. It never infers training quality, and the horse-feel value is explicitly attributed to the rider, not derived. This is exactly the honesty the service promises.

5.3 Horse Records And Documents

LAYER	DETAIL
Evidence	Horse switcher, timeline, record sheets, uploaded passport/vet/lab documents
Rider actions	Select horse → open timeline → add a record → attach a document → return later and find it
Frontstage	Loading, empty, error, offline, permission-denied states; optimistic updates with rollback; upload progress, cancel, retry
Backstage	Real Supabase CRUD for passport, vet, lab, vaccination, dental, farrier, nutrition, care, notes. Upload path: <code>create-upload-ticket</code> → direct private upload → <code>complete-upload</code> (owner check, ticket expiry, path ownership, size, MIME, magic bytes, SHA-256) → scanner adapter → active media reference. Orphans cleaned every 5 minutes
Support	△ Nobody handles "my document disappeared"

Status. This is the **only journey where implementation, deployment, configuration, and internal rollout have all converged.** It works for one named internal user.

Failure points. △ Not proven on hosted accounts — the Horse→Record→upload→restart journey and Rider A/B/C isolation have not been run against the live project. △ Malware scanner not contracted. △ EXIF/GPS not stripped — a stable photo carries its coordinates.

5.4 Academy And Ralf

LAYER	DETAIL
Evidence	One personalized path, searchable lessons, Ralf conversation
Rider actions	Read a recommendation and its reason → open a lesson → ask Ralf → act on the answer

LAYER	DETAIL
Frontstage	Explainable recommendation reasons; editable Ralf context; confidence shown only where it aids a decision; health suggestions phrased as consider a vet review with the rider-entered observations that triggered them
Backstage	Persisted conversations with ownership RLS; every response carries task type, factual context used, confidence reason, safety class, escalation rule, provider version; ⚠ lesson progress is local only
Support	⚠ No safety reviewer for escalation events

Failure points. ⚠ No AI provider contracted — and selecting one requires an explicit decision that rider prompts and horse context are **not** used as training data. ⚠ Safety escalations have nowhere to go. ⚠ Lesson progress resets on reload.

Design strength: confidence is derived from structured context coverage, never accepted from model prose. Ralf cannot diagnose, prescribe, replace a coach, or confirm equipment fit. The saddle-fit screening returns qualitative states — `Worth inspecting`, `Measurements needed`, `Ask the seller` — and routes to a qualified saddler. An arbitrary fit percentage was deliberately removed. This is the clearest example in the codebase of the service refusing to overclaim.

5.5 Club

LAYER	DETAIL
Evidence	Feed, post detail, comments, reactions, report and block controls
Rider actions	Read → post → attach a photo or ride → comment → report something harmful
Frontstage	Optimistic actions with rollback; visible moderation and deleted-content states
Backstage	Posts, comments, reactions, reports, moderation jobs, and RLS all exist; ⚠ the UI is wired to a mock API and local reaction counters
Support	⚠ Moderation queue with no moderator

Failure points. ⚠ No moderation provider configured. ⚠ No human moderation process, deadline, or escalation path. ⚠ Every authenticated account can read all profiles **including location** — for an equestrian product this discloses where a valuable animal is kept. ⚠ Public media publishing is blocked on the missing EXIF processor.

Service rule: a report button that opens a queue nobody reads is worse than no report button, because it converts the rider's effort into false assurance. Club must not activate until a named moderator, a review deadline, and a sanction/appeal path exist.

5.6 Marketplace — Buyer

LAYER	DETAIL
Evidence	Listings, condition evidence, seller trust signals, fit screening, protection explanation
Rider actions	Browse → inspect → screen fit → message the seller → (blocked) buy
Frontstage	Real public catalog reads, uncoupled from messaging, with no production seed fallback; honest missing-measurement states; checkout is an explicit read-only preview
Backstage	Participant RLS, blocking, reports, message nonces, outbox, Realtime; strong order state machine and webhook code

LAYER	DETAIL
Support	⚠ No dispute adjudicator, no support ownership

Failure points. ⚠ Messaging is unproven on two devices. ⚠ Checkout is disabled with **zero supported buyer and seller countries** — correctly, given no Stripe Connect, tax, shipping, legal terms, or dispute operations.

Language rule: the payment model is protected marketplace payment with delayed seller transfer. It must **never** be described as escrow without a licensed escrow provider. Similarly, message privacy is TLS plus participant RLS with server-side readability for fraud and moderation — never "end-to-end encrypted", and never with delivery, presence, or typing guarantees.

5.7 Marketplace — Seller

LAYER	DETAIL
Evidence	Dashboard, listings, orders, revenue, payout state, verification status
Rider actions	Verify identity → create a listing → answer buyers → ship → get paid
Frontstage	⚠ All sample/read-only
Backstage	seller-onboarding, review-listing, release-order, resolve-dispute, stripe-webhook implemented; ⚠ none deployed
Support	⚠ Nobody owns payout reconciliation, KYC review, or prohibited-goods enforcement

Gate: every item in `docs/EQUINA_CHECKOUT_ACTIVATION_CHECKLIST.md` must have an accountable owner and staging evidence. Country expansion is a new approval, not a percentage rollout.

5.8 Account Controls — Export And Deletion

LAYER	DETAIL
Evidence	Export download link; deletion confirmation with a stated effective date
Rider actions	Request my data → receive it → verify it is complete → optionally delete my account → optionally change my mind
Frontstage	Recent-authentication requirement; 14-day grace period ; cancel before the effective date; signed export link with stated expiry
Backstage	Deletion Edge Functions with audit trail; direct Data API mutation correctly revoked; export cleanup has leases; ⚠ deletion worker implemented but not deployed
Support	⚠ Nobody handles a failed export or a contested deletion

Failure point. ⚠ `request-data-export` is **synchronous and unpaginated**. It omits parts of Club, media metadata, and seller/listing/order/dispute/review/report history, and has never been tested above 1,000 rows per domain. An export that silently omits domains is worse than none, because it presents itself as complete. It must become an asynchronous, paginated, versioned, privately-stored job before `accountSettings` activates.

Design strength: the 14-day grace period with an explicit cancel path is genuinely good service design. Account deletion is the most irreversible thing a rider can do, and the service gives them two weeks to change their mind.

6 Moments Of Truth

The six points where trust is won or lost:

#	MOMENT	RIDER 'S QUESTION	CURRENT ANSWER
M1	Verification email arrives	"Is this real?"	⚠ Often never arrives
M2	First ride recap	"Did it keep what I did?"	⚠ No — it resets
M3	Reopening a record days later	"Is my horse's history safe here?"	✓ Yes, for the internal user
M4	Ralf answers a health question	"Is this responsible?"	✓ Yes by design; ⚠ no provider, no reviewer
M5	Reporting harmful content	"Does anyone read this?"	⚠ No
M6	Requesting data or deletion	"Am I actually in control?"	⚠ Grace period yes; export incomplete

M1 and M2 are the two that decide whether the product has any users at all. Both are gated on work that is cheap relative to its impact: configure SMTP, and persist ride entries through the repository layer.

7 Channel Design

CHANNEL	PURPOSE	RULES	STATUS
In-app	Primary surface	One primary action per view; every metric names its source; disabled actions state their boundary	Active
Email	Verification, recovery, export links	Branded templates; SPF/DKIM/DMARC; delivery monitoring	⚠ Not configured
Push	Human messages, order changes, horse reminders, academy reminders	Redacted payloads only — New message about [listing], route, conversation ID. Never message bodies, horse health, AI prompts, email, or payment data. Quiet hours enforced; badges only for critical updates	⚠ Not activated
Support	Human recourse	⚠ Does not exist — no address, no queue, no SLA	⚠

The push design is notably disciplined: category preferences, quiet hours with a timezone, redacted previews, sign-out revocation, and invalid-token cleanup are all specified. The rule against notification bait is written into the psychology contract, not just implied.

8 The Missing Operating Model

Substantial backend machinery exists for moderation, reporting, sanctions, disputes, deletion, and safety escalation. **None of it has a human behind it.** This table is the work that has to happen before the corresponding flags may move — and it is not engineering work.

QUEUE	TRIGGERED BY	NEEDS	OWNER	SLA
Content moderation	clubPublishing, clubInteractions, listingCreation	Reviewer, review deadline, sanction ladder, appeal path	Unassigned	Undefined

QUEUE	TRIGGERED BY	NEEDS	OWNER	SLA
Abuse reports	any social surface	Triage rules, block/report handling, repeat-offender policy	Unassigned	Undefined
Safety escalation	coachChat	Reviewer able to judge health-flag events	Unassigned	Undefined
Deletion and export	accountSettings	Failure handling, contested-deletion path	Unassigned	Undefined
Disputes and refunds	shopTransactions	Adjudicator, evidence rules, refund authority	Unassigned	Undefined
Platform on-call	any expansion	Alert routing, runbook, escalation	Unassigned	Undefined

Recommendation: no capability flag may be enabled until its queue in this table has a named owner and a stated response time. At current scale — one internal user — a single named person covering all six queues on a next-business-day commitment is sufficient and honest. What is not acceptable is enabling a report button with no reader.

9 Service Readiness Matrix

JOURNEY	DESIGNED	BUILT	DEPLOYED	OPERABLE	VERDICT
Onboarding	✓	✓	partial	⚠ no email delivery	Blocked
Daily ride loop	✓	prototype	—	n/a	Not persistent
Horse records	✓	✓	✓	internal only	Closest to ready
Academy	✓	prototype	—	n/a	Not persistent
Ralf	✓	✓	—	⚠ no provider, no reviewer	Blocked
Club	✓	backend only	—	⚠ no moderator	Blocked
Marketplace browse	✓	✓	✓	✓	Read-only, fine
Marketplace messaging	✓	✓	—	⚠ no two-device proof	Blocked
Seller	✓	backend only	—	⚠ no owner	Blocked
Checkout	✓	backend only	—	⚠ no legal, finance, support	Correctly disabled
Export / deletion	✓	partial	—	⚠ export incomplete	Blocked

"Operable" is the column this document adds, and it is the one that fails most often. The pattern across the matrix is consistent: **Equina's engineering is well ahead of its operations.**

10 Service Recommendations

Immediate — restores the service's ability to have users at all

1. Configure custom SMTP with domain authentication, branded templates, and delivery monitoring. Without it there is no service, only a demo. (M1)

2. Persist ride entries through the repository layer. The daily loop is the retention engine and it currently forgets. (M2)

Before any social or account surface activates

3. Name one person as moderator, support, and safety reviewer, with a next-business-day response commitment. Publish a support contact in-app.
4. Replace broad profile reads with a public projection plus explicit visibility choices. Location exposure is a physical-security concern for horse owners, not only a privacy one.
5. Replace the synchronous export with an async, paginated, versioned job.
6. Build the EXIF/normalization worker before any media becomes public.

Before commerce is considered at all

7. Treat checkout as a separate regulated launch with named legal, finance, and support owners, per the activation checklist. Do not schedule it on the product track.

Ongoing

8. Keep the action registry, the production claim registry, and the release gate. They are the mechanism that makes the honesty promise enforceable rather than aspirational, and they are the most valuable service asset the project has.

Related documents: `EQUINA_IMPACT_ASSESSMENT.md` , `EQUINA_TECHNICAL_DESIGN.md` ,
`EQUINA_ACTION_REGISTRY.md` , `EQUINA_EXPERIENCE_STATE_CONTRACT.md` ,
`EQUINA_INTERACTION_CONTRACT.md` , `EQUINA_DESIGN_CONTRACT.md` ,
`EQUINA_FEATURE_FLAG_POLICY.md` , `EQUINA_CHECKOUT_ACTIVATION_CHECKLIST.md` .